

MODEL CONTEXT PROTOCOL · KNOWLEDGE GRAPH MEMORY

mnemex

v0.1.0

Persistent memory for AI assistants, stored as a [Neo4j knowledge graph](#) instead of a flat chunk index. Claims, the people who made them, and the decisions that supersede each other are all first-class nodes and edges.

- Neo4j Aura
- Nosana GPU · embeddings + Qwen2.5-7B
- Daytona sandboxes
- TypeScript · MCP SDK
- Next.js 16 demo UI

THE PROBLEM

Vector RAG remembers text. It forgets **structure**.

Chop the transcript, embed the chunks, return the nearest neighbours.
That answers “what was said about X” and fails at everything built on
relationships.

FAILURE 01

Who held which position

A chunk containing an objection does not record who made it. Attribution dies the moment the text is flattened.

FAILURE 02

What is still true

A reversed decision and the decision that reversed it are two similar chunks. Search returns both, with no signal which survived.

FAILURE 03

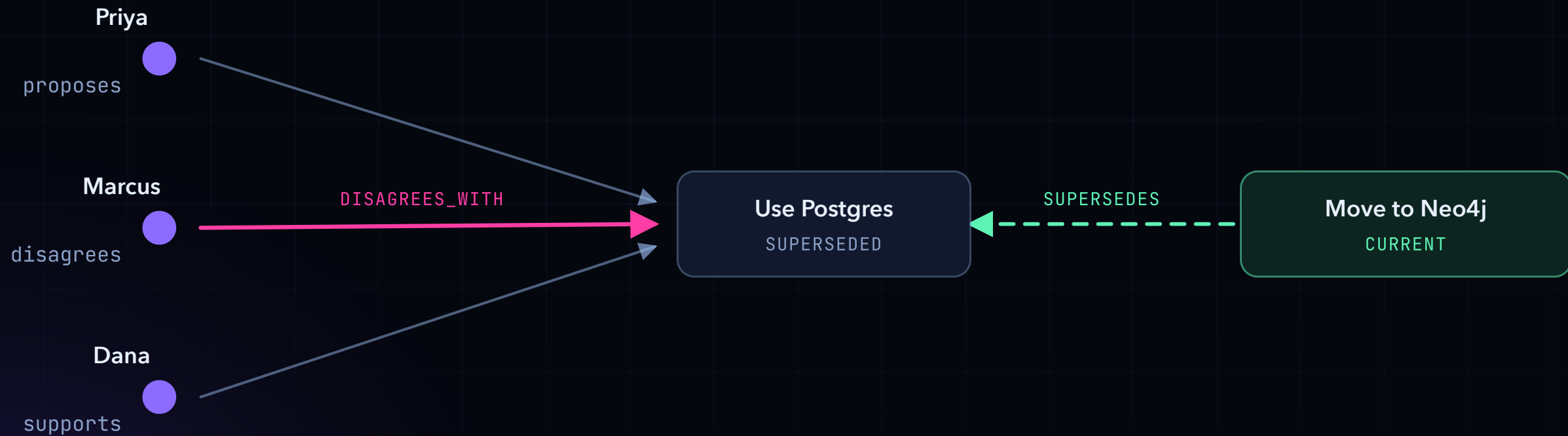
How things connect

“What did we decide, and who disagreed” is a two-hop traversal. No amount of top-k tuning turns it into one.

The failure is not retrieval quality. The storage format threw the relationships away before retrieval ever ran.

THE IDEA

Store the argument, not the transcript.



Every one of those labels is a real edge in Neo4j. Recall is a graph traversal seeded by a vector search, not a similarity ranking on its own.

WHAT IT DOES

Four tools over stdio. One config entry.

TOOL 01 • WRITE

remember

One speaker turn: session, speaker, claims, topic, and the stance taken on a decision. Resolves the person, deduplicates restated claims, detects supersession, commits atomically.

TOOL 02 • READ

recall

A natural-language question. Returns each decision with its status, everyone who supported or objected, the claim each made, and the supersession chain both ways.

TOOL 03 • READ

timeline

Takes a topic. Returns its decision history in order, showing what superseded what and who dissented at each step.

TOOL 04 • ESCAPE HATCH

analyze

Assistant-authored Cypher for the questions the fixed traversals miss. Read-only, executed inside a sandbox whose egress reaches nothing but the database.

```
// the assistant supplies structure; mnemex does not scrape it from raw transcripts
remember({ sessionId: "s1", speaker: "Marcus", topic: "storage engine",
  claims: ["Our access pattern is almost entirely graph traversal."],
  decision: { statement: "Use Postgres as the primary datastore",
    stance: "disagrees" } })
```

Every response carries a **degraded** array. Empty on the happy path, otherwise it names the subsystem running on a fallback.

GRAPH SCHEMA

Five node labels. Eight kinds of edge.

NODES

Session	One conversation. id, title, startedAt
Person	A participant. id, name
Topic	Subject matter. id, name, slug
Decision	An outcome. statement, status, decidedAt, embedding
Claim	One atomic thing said. text, saidAt, embedding

status ∈ { current, superseded, open }

EDGES

STATED_BY	Claim → Person. Attribution. Every position has an owner.
SUPPORTS	Claim → Decision. Agreement with an outcome.
DISAGREES_WITH	Claim → Decision. The edge flat chunks cannot represent.
SUPERSEDES	Decision → Decision. Enables “what is still true”.
ABOUT	Claim or Decision → Topic. Subject linkage.
IN_SESSION	Claim → Session. Provenance.
DECIDED_IN	Decision → Session. Provenance.
PARTICIPATED_IN	Person → Session. Roster.

Two cosine vector indexes, `claim_embedding` and `decision_embedding`, at 768 dimensions.
Five uniqueness constraints. All of it created by one bootstrap script.

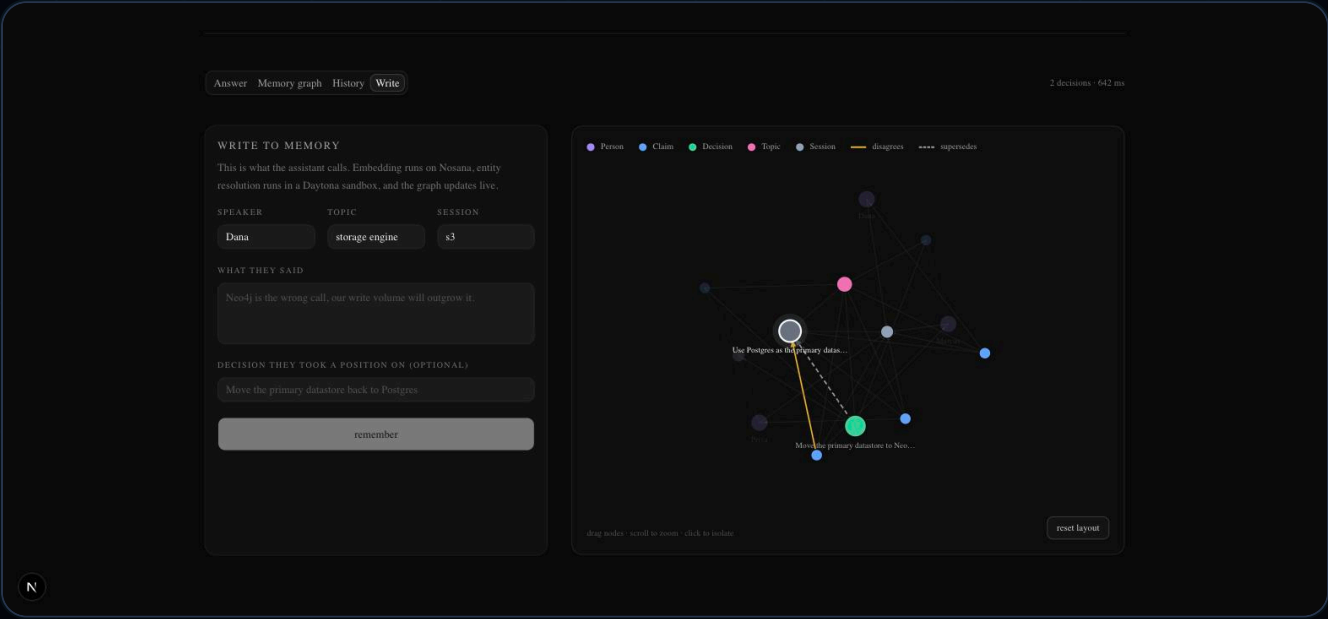
— WRITE PATH

Untrusted text never plans its own merge.

- 01 **Embed on Nosana** – claims and the decision go to a GPU-served BGE model over an OpenAI-compatible route.
- 02 **Fetch candidates** – both vector indexes return the 25 nearest claims and decisions as a bounded candidate set.
- 03 **Plan the merge in a Daytona sandbox** – entity resolution and supersession detection run isolated, then the plan is validated against the candidate set it was given.
- 04 **Commit** – people, claims, stance edges and the supersession edge land in Neo4j.

LIVE WRITE, GRAPH UPDATES IN PLACE

A claim only merges into an existing one at cosine 0.97 or above. The threshold is deliberately conservative: duplicate nodes are recoverable, a wrong merge attributes an argument to the wrong person.



— READ PATH

The vector index finds where to start. Cypher collects the answer.

STEP 1 • SEED

The question is embedded once, then queried against both vector indexes. This picks the entry points, and nothing more.

STEP 2 • TRAVERSE

From each seed, Cypher walks out to every claim that supports or opposes the decision, to the person behind each claim, and along the supersession chain in both directions.

STEP 3 • RANK

Decisions that still stand sort first. What is currently true is what the caller asked for.

what did we decide about the storage engine, and who disagreed?

storage engine

recall

what did we decide about the storage engine, ...

who objected to using Postgres?

what is the current decision about the datast...

Answer

Memory graph

History

Write

2 decisions · 642 ms

CURRENT

Move the primary datastore to Neo4j

overrides Use Postgres as the primary datastore

SUPPORT

Marcus

The traversal queries are now eight joins deep and the latency is unacceptable.

Priya

The join depth argument is convincing. I withdraw my earlier position.

SUPERSEDED

Use Postgres as the primary datastore

overruled by Move the primary datastore to Neo4j

DISSENT

Marcus

Our access pattern is almost entirely graph traversal, and Postgres will make that painful.

SUPPORT

Priya

Postgres gives us transactional guarantees we already understand.

Dana

Operationally Postgres is the thing we can actually run on call.

N

LIVE MEMORY

Click a node, get its typed connections.

what did we decide about the storage engine, and who disagreed?

storage engine

recall

what did we decide about the storage engine, ...

who objected to using Postgres?

what is the current decision about the datast...

Answer

Memory graph

History

Write

2 decisions · 642 ms

Person

Claim

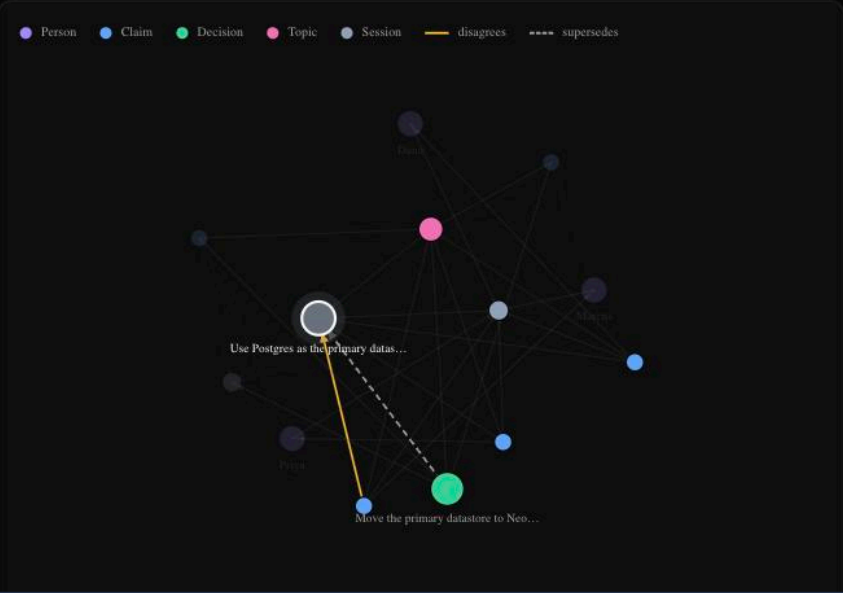
Decision

Topic

Session

disagrees

supersedes



INSPECTOR

refresh

Decision · superseded

Use Postgres as the primary datastore

CONNECTIONS (6)

← SUPPORTS

Postgres gives us transactional guarantees we already understand.

→ ABOUT

storage engine

→ DECIDED_IN

Storage layer review

← DISAGREES_WITH

Our access pattern is almost entirely graph traversal, and Postgres will make that painful.

← SUPPORTS

Operationally Postgres is the thing we can actually run on call.

← SUPERSEDES

Move the primary datastore to Neo4j

13

NODES IN MEMORY

30

EDGES

1

DISSENT EDGE

1

SUPERSESSION

6

CONNECTIONS ON THIS NODE

TECHNOLOGY

Four platforms, each on the critical path.

SYSTEM OF RECORD

Neo4j Aura

Nodes, edges and both vector indexes. Reached through the official driver with lossless integers disabled and a ten-second connection ceiling.

NOSANA · EMBEDDINGS

BGE-base-en-v1.5

Served by vLLM on a decentralised GPU market over an OpenAI-compatible route. The identical checkpoint runs locally through ONNX when the endpoint is gone.

NOSANA · REASONING

Qwen2.5-7B-Instruct

The assistant itself, on an A6000, with automatic tool choice and the Hermes tool-call parser. It is the thing calling mnemex, so the whole loop runs on rented GPU.

ISOLATION

Daytona

A disposable sandbox on both paths: the merge planner on write, and assistant-authored Cypher on read, egress-limited to the database host alone.

TRANSPORT

MCP SDK

Four tools over a stdio transport. The Zod schemas are the tool contract and the runtime validation at once, so the two cannot drift apart.

DEMO SURFACES

Next.js 16 · React 19

Route handlers import the compiled package rather than reimplementing it, so the page exercises the MCP code path. Graph drawn with d3-force.

TypeScript in strict mode throughout. Zod validates every boundary. The golden suite runs end to end against a live Aura instance, not a mock.

BUILD STATUS

Two of five agents in. Both landed working infrastructure, not scaffolding.

VERIFIED · TOOL 04

analyze is live

A real analytical query ran generated Cypher inside a Daytona sandbox against a read-only credential, with nothing degraded. Writes are refused in under half a second, without ever creating a sandbox. All four tools are registered.

Analyze was given its own timeout, separate from the write path: it has no fallback by design, so waiting out a slow sandbox is the only useful response, where a write should degrade in process quickly.

```
// the full stance breakdown, per person.
// no fixed traversal returns this shape.
Priya      SUPPORTS      2
Marcus     DISAGREES_WITH 1
Dana       SUPPORTS      1
Marcus     SUPPORTS      1
```

VERIFIED · NOSANA

the chat model is live

Qwen2.5-7B-Instruct, with tool calling verified rather than assumed. It returned a well-formed tool call for recall with parseable arguments, and named the dissenter correctly when the result was fed back.

CORRECTIONS

confidential:false	Necessary, not sufficient. A deployment also needs autostart:true, because the separate start call returns an undiagnosable 500.
online	Not a readiness signal. The field reads false while the endpoint is already serving traffic.

4.36 / 10

CREDITS RESERVED

768

EMBED DIMS · LIVE

Qwen 7B

CHAT LIVE · TOOL CALLS

a6000

GPU MARKET · 3090S BUSY

Three agents still running: the AgentCanvas chat UI, the graph selection bug, and the UI gap audit.

— FAILURE MODES AND SETUP

Nothing remote is allowed to hang the assistant.

Nosana unreachable	→	local ONNX model, <code>embeddings:local</code>
Daytona, on the write path	→	in-process merge planning, <code>normalizer:inprocess</code>
Daytona, on the read path	→	analyze switches off, <code>analyze:disabled</code> . Model-authored Cypher never runs unsandboxed.
Neo4j unreachable	×	the write throws. Memory that silently vanishes is worse than an error.

3s	15s	30s	642ms
EMBEDDING BUDGET	WRITE SANDBOX	ANALYZE SANDBOX	RECALL, LOCAL EMBED

```
# set up
git clone https://github.com/Sizbei/mnemex.git
npm install && cp .env.example .env
npm run bootstrap # constraints + vector indexes
npm run build

# see it work
npm run seed -- --reset
npm run demo

# deploy the model, then wire the assistant up
npm run deploy:chat
claude mcp add mnemex -- node "$(pwd)/dist/src/server.js"
```

PASS THE ENVIRONMENT IN THE MCP CONFIG: THE CLIENT LAUNCHES THE SERVER FROM AN ARBITRARY DIRECTORY.